FONDAMENTI DI MECCANICA

Simulazione numerica e controllo di un pendolo inverso mediante Reinforcement Learning

James Ray Bamber **Giovanni Matteazzi**

Scuola Superiore Sant'Anna, Pisa

17 aprile 2026

Indice

1 Introduzione **3**

2 Trattazione analitica **3**
2.1 Descrizione del modello 3
2.2 Derivazione dell'equazione del moto 4

3 Controllo **6**
3.1 Q-Learning 7
3.2 Deep Q-Network 8
3.3 Dati raccolti 10

1 Introduzione

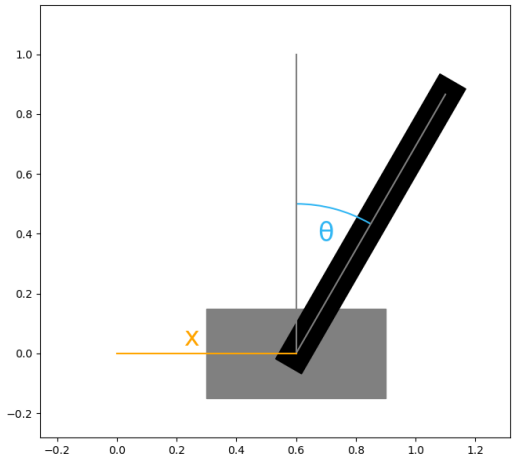
Il pendolo invertito su carrello (*cart-pole*) è un sistema classico di meccanica: un'asta rigida, incernierata alla sommità di un carrello mobile, deve essere mantenuta in equilibrio instabile nella posizione verticale mediante una forza orizzontale applicata al carrello stesso.

Il progetto si articola in due fasi. Nella prima, si ricava analiticamente il modello dinamico del sistema attraverso il formalismo Lagrangiano, e se ne realizza una simulazione numerica con integratori del tipo Runge-Kutta.

Nella seconda fase, il modello viene utilizzato come ambiente per addestrare un agente di controllo autonomo tramite due tecniche di Reinforcement Learning: il Q-Learning, che opera su una discretizzazione dello spazio degli stati, e la Deep Q-Network, che approssima la funzione di valore mediante una rete neurale e consente di trattare direttamente lo spazio degli stati continuo. L'obiettivo del controllo è mantenere l'angolo dell'asta entro $\pm 12°$ dalla verticale e il carrello entro $\pm 2.4\,\mathrm{m}$ dall'origine.

2 Trattazione analitica

2.1 Descrizione del modello



Il pendolo inverso è costituito da un **carrello**, considerato come un punto materiale di massa m_1, libero di muoversi lungo un asse, e da un'**asta** omogenea di massa m_2 e lunghezza l, libera di ruotare attorno al carrello. Per bilanciare in verticale l'asta è possibile applicare una forza di modulo F sulla base del carrello, orientata, a seconda dello stato del sistema, verso destra o verso sinistra.

Uno **stato** è dato dalle 4 variabili continue che descrivono il problema: la coordinata x del carrello e la sua velocità $\dot{x}$, l'angolo che l'asta forma con la

verticale θ e la velocità angolare $\dot{\theta}$. Una volta note queste grandezze, le posizioni dei centri di massa di carrello e asta sono date da:

$$\begin{cases} x_1 = x \\ y_1 = 0 \\ x_2 = x + \frac{l}{2} sin\theta \\ y_2 = \frac{l}{2} cos\theta \end{cases} \tag{1}$$

2.2 Derivazione dell'equazione del moto

Possiamo ora dedicarci alla risoluzione dinamica del sistema per mezzo della equazione di Eulero-Lagrange, applicabile una volta trovata l'espressione per la Lagrangiana $L(x, \dot{x}, \theta, \dot{\theta})$.

Si trova facilmente che l'**energia cinetica** del carrello vale:

$$T_1 = \frac{1}{2} m_1 v_1^2 = \frac{1}{2} m_1 \dot{x}^2 \tag{2}$$

Poiché invece l'asta, oltre a traslare, ruota, dobbiamo separare le componenti traslatoria e rotatoria della sua energia cinetica. Per trovare quella traslatoria deriviamo il sistema di coordinate (1) per trovare la velocità del suo centro di massa:

$$\begin{cases} \dot{x_2} = \dot{x} + \frac{l}{2} \dot{\theta} cos\theta \\ \dot{y_2} = -\frac{l}{2} \dot{\theta} sin\theta \end{cases}$$

Il contributo traslatorio è quindi:

$$\begin{aligned} T_{2,t} &= \frac{1}{2} m_2 v_2^2 = \frac{1}{2} m_2 (\dot{x_2}^2 + \dot{y_2}^2) \\ &= \frac{1}{2} m_2 \left(\dot{x}^2 + \frac{l^2}{4} \dot{\theta}^2 \cos^2\theta + l\dot{x}\dot{\theta} \cos\theta + \frac{l^2}{4} \dot{\theta}^2 \sin^2\theta \right) \\ &= \frac{1}{2} m_2 \left(\dot{x}^2 + \frac{l^2}{4} \dot{\theta}^2 + l\dot{x}\dot{\theta} \cos\theta \right) \end{aligned} \tag{3}$$

Per quanto riguarda invece la componente rotatoria è necessario trovare il momento di inerzia I. Essendo l'asta omogenea, essa avrà una densità lineare $\lambda = m_2/l$. Considerando una somma infinita di segmenti infinitesimi dx di massa dm calcoliamo

$$\begin{aligned} I &= \int_{pole} x^2 dm = \int_0^l x^2 \lambda dx = \lambda \int_0^l x^2 dx = \\ &= \frac{m_2}{l} \left[\frac{x^3}{3} \right]_0^l = \frac{m_2}{l} \left(\frac{l^3}{3} \right) = \frac{1}{3} m_2 l^2 \end{aligned}$$

Allora, abbiamo

$$T_{2,r} = \frac{1}{2} I \dot{\theta}^2 = \frac{1}{2} \cdot \frac{1}{3} m_2 l^2 \dot{\theta}^2 = \frac{1}{6} m_2 l^2 \dot{\theta}^2 \tag{4}$$

Quindi, per l'asta, sommando i contributi (3) e (4), si ha

$$T_2 = \frac{1}{2}m_2\dot{x}^2 + \frac{1}{2}m_2 l\dot{x}\dot{\theta}cos\theta + \frac{7}{24}m_2 l^2\dot{\theta}^2 \tag{5}$$

Oss: Per un simile sistema formato dal carrello e da un punto materiale di massa m_2 che ruota a distanza l, il coefficiente del termine contenente $\dot{\theta}^2$ sarebbe stato pari a $\frac{1}{2}$, cioè superiore a quello ottenuto nel caso di studio. Questo conferma un'intuizione fisica: il modello con l'asta, nonostante porti a calcoli leggermente più laboriosi, è più stabile e più semplice da controllare in un task non banale come stabilizzare l'elemento rotante.

L'**energia potenziale** del sistema è solo quella gravitazionale dell'asta, quindi, usando la (1), si ottiene

$$U = \frac{1}{2}m_2 glcos\theta \tag{6}$$

Per le (2), (5), (6) possiamo ora calcolare

$$\begin{aligned} L(x,\dot{x},\theta,\dot{\theta}) &= T - U = T_1 + T_2 - U \\ &= \frac{1}{2}m_1\dot{x}^2 + \frac{1}{2}m_2\dot{x}^2 + \frac{1}{2}m_2 l\dot{x}\dot{\theta}cos\theta + \frac{7}{24}m_2 l^2\dot{\theta}^2 - \frac{1}{2}m_2 glcos\theta \end{aligned}$$

A questo punto, possiamo applicare l'**equazione di Eulero-Lagrange** indipendentemente per entrambe le variabili, x e θ. Inoltre, poiché sul carrello agisce orizzontalmente la forza generalizzata non conservativa F, nella prima equazione - e solo in questa, F non è una coppia diretta sull'asta - non dobbiamo dimenticare il termine a destra dell'uguale.

Usiamo allora

$$\frac{d}{dt}\left(\frac{\partial L(q,\dot{q})}{\partial \dot{q}}\right) - \frac{\partial L(q,\dot{q})}{\partial q} = \sum F_i.$$

Rispetto a x, prendendo F con il suo segno ($-$ se verso sinistra, $+$ verso destra):

$$\frac{d}{dt}\left(m_1\dot{x} + m_2\dot{x} + \frac{1}{2}m_2 l\dot{\theta}cos\theta\right) - 0 = F$$

$$(m_1 + m_2)\ddot{x} + \frac{1}{2}m_2 l\ddot{\theta}cos\theta = F \tag{7}$$

Invece, con riferimento a θ:

$$\frac{d}{dt}\left(\frac{1}{2}m_2 l\dot{x}cos\theta + \frac{7}{12}m_2 l^2\dot{\theta}\right) - \left(-\frac{1}{2}m_2 l\dot{x}\dot{\theta}sin\theta + \frac{1}{2}m_2 glsin\theta\right) = 0$$

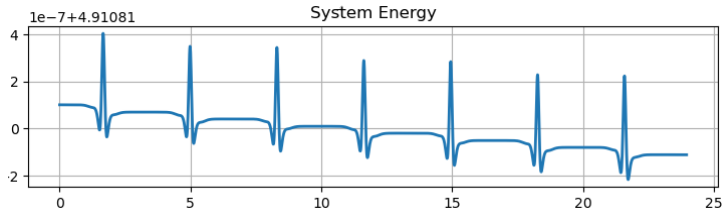
$$\frac{1}{2}m_2 l\ddot{x}cos\theta + \frac{7}{12}m_2 l^2\ddot{\theta} - \frac{1}{2}m_2 l\dot{x}\dot{\theta}sin\theta + \frac{1}{2}m_2 l\dot{x}\dot{\theta}sin\theta - \frac{1}{2}m_2 glsin\theta = 0$$

$$\frac{7}{6}l\ddot{\theta} + \ddot{x}cos\theta - gsin\theta = 0 \tag{8}$$

Le equazioni (7) e (8) costituiscono un sistema lineare in $\ddot{x}$ e $\ddot{\theta}$. Risolvendo esplicitamente si ottiene:

$$\begin{cases} \ddot{x} = \dfrac{7F + \dfrac{7}{2} m_2 l\, \dot{\theta}^2 \sin\theta - 3\, m_2\, g \cos\theta \sin\theta}{7(m_1 + m_2) - 3\, m_2 \cos^2\theta} \\ \ddot{\theta} = \dfrac{6}{7l}\big(g \sin\theta - \ddot{x} \cos\theta\big) \end{cases} \tag{9}$$

Date delle condizioni iniziali $(x_0, \dot{x}_0, \theta_0, \dot{\theta}_0)$, le equazioni (9) consentono di propagare numericamente passo passo lo stato del sistema tramite un integratore, per esempio RK4, ricostruendo deterministicamente la traiettoria del pendolo inverso.



Energia del sistema quando la forza F è nulla, integrato con RK4.

3 Controllo

Affrontiamo ora il problema del **controllo** del sistema, con l'obiettivo di mantenere l'asta in posizione verticale per il maggior tempo possibile.

Formalmente, richiediamo che l'angolo dell'asta rispetto alla verticale rimanga limitato, ossia $|\theta| < 12°$, e che il carrello non si allontani eccessivamente dall'origine, imponendo $|x| < 2.4$. L'episodio è considerato terminato nel momento in cui una di queste condizioni viene violata.

Per affrontare questo problema adottiamo un approccio di **Reinforcement Learning**, in cui l'agente apprende attraverso l'interazione diretta con l'ambiente. In particolare, l'apprendimento è guidato da un segnale di **reward**: azioni che contribuiscono a mantenere l'equilibrio producono ricompense positive, mentre il fallimento comporta una penalizzazione.

L'**agente** interagisce con l'ambiente scegliendo, a ogni passo temporale, tra un insieme discreto di azioni: applicare una forza verso sinistra (azione 0) oppure verso destra (azione 1). Proprio perché lo spazio delle azioni è discreto (ha 2 valori possibili), risultano particolarmente adatti metodi come il Q-Learning e la sua estensione con reti neurali profonde, il Deep Q-Network (DQN), che verranno analizzati nei paragrafi successivi.

3.1 Q-Learning

Il **Q-Learning** è un algoritmo di Reinforcement Learning che mira a stimare la funzione $Q(s, a)$, ovvero il valore atteso del reward cumulativo (che conta cioè i contributi di tutti gli step futuri che conducono a uno stato terminale), ottenuto eseguendo l'azione a nello stato s e seguendo successivamente una politica ottimale.

Poiché lo spazio degli stati del sistema è continuo, è necessario introdurre una **discretizzazione** per poter rappresentare la funzione Q tramite una tabella finita. Nel nostro caso, lo stato è descritto da quattro variabili:

$$s = (x, \theta, \dot{x}, \dot{\theta})$$

che vengono discretizzate nei seguenti intervalli:

- $x \in [-2.4, 2.4]$ in 7 stati
- $\theta \in [-12°, 12°]$ in 25 stati
- $\dot{x} \in [-2, 2]$ in 7 stati
- $\dot{\theta} \in [-3, 3]$ in 25 stati

La funzione Q è quindi rappresentata come una tabella multidimensionale:

$$Q \in \mathbb{R}^{7\times25\times7\times25\times2}$$

dove l'ultima dimensione corrisponde alle due azioni possibili.

A ogni passo temporale del ciclo di allenamento, la stima di $Q(s, a)$ viene aggiornata secondo la regola:

$$Q(s, a) = Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \tag{10}$$

dove α è il learning rate, γ è il fattore di sconto, r è il reward immediato, s' è lo stato successivo.

L'**algoritmo** che porta a convergenza la tabella prevede che, per ogni episodio:

- si inizializza lo stato s^1;
- fino al raggiungimento di uno stato terminale o di un numero massimo di passi:
  - si seleziona un'azione a tramite una politica ϵ-greedy[2];
  - si esegue l'azione e si osservano il nuovo stato s^3 e la ricompensa r^4;
  - si aggiorna $Q(s, a)$ come in (10);
  - si pone s' come nuovo stato;
- si riduce ϵ.

Note implementative:

1. Lo stato iniziale viene campionato da una distribuzione che evolve nel tempo: nelle fasi iniziali sono presi stati vicini all'equilibrio, progressivamente stati sempre più distanti e "difficili" (curriculum learning).
2. si sceglie con probabilità ϵ una azione random (exploration), con probabilità $1 - \epsilon$ l'azione ottimale data dalla tabella attuale (exploitation).
3. lo stato ottenuto va discretizzato.
4. il reward è definito in modo continuo come

$$r = 1 - \frac{|\theta|}{\theta_{\max}} - 0.05\frac{|\dot{\theta}|}{\dot{\theta}_{\max}} - 0.02\frac{|x|}{x_{\max}}.$$

3.2 Deep Q-Network

Nonostante il successo nell'obiettivo, il Q-Learning presenta limiti evidenti quando lo spazio degli stati è ampio o ha alta dimensionalità: una discretizzazione più fitta porta a problemi computazionali, se più lasca introduce perdita di informazione.

Per superare questi limiti si introduce il **Deep Q-Network** (DQN), che sostituisce la tabella $Q(s, a)$ con una funzione approssimata da una Deep Neural Network. In questo modo è possibile lavorare direttamente su stati continui, evitando *in toto* il problema della discretizzazione.

Questa rete neurale prende in input lo stato **continuo** $s \in \mathbb{R}^4$ e restituisce in output un vettore di dimensione pari al numero di azioni, contenente i valori Q associati a ciascuna di esse.

Nel nostro caso, la rete è composta da due strati nascosti da 64 neuroni ciascuno, utilizza una funzione di perdita di tipo MSE, ReLU come attivazione e l'ottimizzatore Adam per l'aggiornamento dei pesi.

Il **target** utilizzato è:

$$y = \begin{cases} r & \text{se lo stato è terminale} \\ r + \gamma \max_a Q_{\theta^-}(s') & \text{altrimenti} \end{cases}$$

dove Q_{θ^-} è la target network, ovvero una copia congelata della rete principale.

Definito il target y, la funzione di errore misura lo scarto tra il valore predetto $Q_\theta(s)$ e il target stesso, permettendo di calcolare, tramite backpropagation, l'aggiornamento dei pesi della rete.

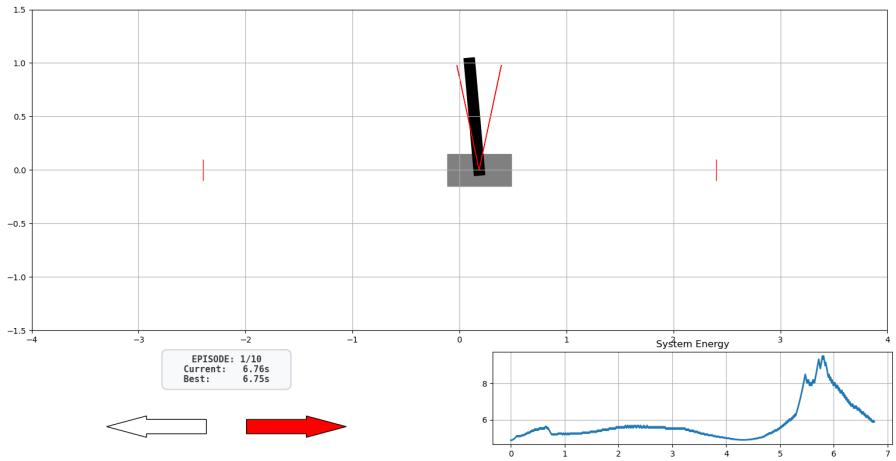
Come accennato, per stabilizzare l'apprendimento si introduce una seconda rete, detta **target network**, i cui pesi vengono aggiornati periodicamente copiando quelli della rete principale. Essa viene utilizzata esclusivamente per

il calcolo del target, evitando che questo vari troppo rapidamente durante il training.

Un ulteriore elemento fondamentale è il **replay buffer**, implementato come una memoria finita che memorizza le transizioni $(s, a, r, s', done)$.

Durante l'addestramento, i pesi della rete vengono aggiornati campionando mini-batch casuali da questa memoria. Questo meccanismo rompe la correlazione temporale tra i campioni e rende il processo di apprendimento più stabile ed efficiente.

Il loop di training è analogo a quello del Q-Learning, con la differenza che l'aggiornamento non avviene direttamente sulla funzione Q, ma sui parametri della rete neurale, tramite backpropagation. L'apprendimento, inoltre, avviene campionando mini-batch casuali di transizioni dal replay buffer, invece di utilizzare esclusivamente l'esperienza più recente.



3.3 Dati raccolti

Sono stati simulati 10 episodi con stato iniziale randomico che varia uniformemente tra:

- $x \in [-0.1, 0.1]$
- $\theta \in [-10°, 10°]$
- $\dot{x} \in [-0.1, 0.1]$
- $\dot{\theta} \in [-0.1, 0.1]$

Nella tabella sono riportati i risultati ottenuti, utilizzando diversi metodi di controllo:

Giocatore	**Miglior Punteggio**	**Punteggio Medio**
Giovanni	12.38	3.42
James	10.28	4.67
Q-learning	20.00	10.93
DQN	20.00	20.00

20.00 è il punteggio massimo, al raggiungimento del quale l'episodio è da considerare concluso.

Tutto il codice per realizzare la simulazione e allenare gli agenti si trova nella seguente repository GitHub:
`https://github.com/jamesbamber/cartpole`.

Bibliografia

[1] E. Hairer, G. Wanner, C. Lubich, *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*, Springer, 2nd edition, 2006.

[2] Giorgio Carlo Buttazzo, *04-Reinforcement learning*, dispense del corso di "Fondamenti di Reti Neurali".

[3] GitHub repository:
`https://github.com/pythonlessons/CartPole_reinforcement_learning/tree/master`.